

VELS



INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI



A LOW-CODE FRAMEWORK FOR CONVERSATIONAL AI ASSISTANTS LEVERAGING RETRIEVAL-AUGMENTED GENERATION AND LLMs

PROJECT DISSERTATION II REPORT

Submitted by

THANIGAIVEL G

Reg.No : 24880107

in partial fulfilment of the award of the degree

of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING

**VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED
STUDIES(VISTAS)**

PALLAVARAM, CHENNAI 600117

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project **“A LOW-CODE FRAMEWORK FOR CONVERSATIONAL AI ASSISTANTS LEVERAGING RETRIEVAL-AUGMENTED GENERATION AND LLMs”** is the bonafide work of **THANIGAIVEL G (24880107)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.S. SRIDEVI, M.E, Ph.D,
Associate Professor,
Head of the Department,
Department of CSE,
VISTAS,
Chennai – 600 117.

SIGNATURE

Dr.S. THIRUMAL M.E, Ph.D
Associate Professor,
Supervisor,
Department of CSE,
VISTAS,
Chennai – 600 117.

Submitted for the project work and viva - voice examination held on
..... at Vels Institute of Science, Technology and Advanced
Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH.**, with great pleasure that I take this opportunity to thank him. I am grateful to our **DIRECTOR, DR.K. KALAIVANI M.E. Ph.D., & HEAD OF THE DEPARTMENT, Dr.S. SRIDEVI, M.E. Ph.D.,** for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day.

I extend our warmest thanks to my **INTERNAL GUIDE, Dr. N KUMAR M.Tech. Ph.D., Dr. S. THIRUMAL M.E. Ph.D.,** who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator **Dr. S. THIRUMAL M.E. Ph.D.,** I would like to express my greatest gratitude to our staff members, librarian and non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my **PARENTS** and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

THANIGAIVEL G - 24880107

ABSTRACT

Conversational AI assistants have evolved rapidly with the rise of large language models (LLMs). However, developing customized assistants that integrate external knowledge, handle domain-specific queries, and maintain cost-efficiency still demands advanced programming skills. To address this, the proposed work introduces a low-code framework for building and deploying conversational AI assistants that leverage Retrieval-Augmented Generation (RAG) and LLMs.

The framework enables non-expert users to design intelligent dialogue systems through a drag-and-drop interface. Developers can connect components such as data ingestion, vector indexing, retriever, and LLM modules without extensive coding. A pipeline based on semantic retrieval augments the LLM context with relevant documents, enabling accurate and verifiable responses.

The proposed system combines a Low-Code Designer (for UI and workflow creation), a RAG Engine (for contextual retrieval), and an LLM Runtime (for natural-language reasoning and response generation). Tools such as LangChain, FAISS/Pinecone, FastAPI, and OpenAI/Hugging Face models are employed. Performance is evaluated using accuracy, latency, and usability metrics. The framework aims to democratize the development of advanced conversational agents.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	12
	1.1 OVERVIEW	12
	1.2 OBJECTIVE	12
	1.3 SCOPE OF THIS PROJECT	13
	1.4 PROBLEMT STATEMENT	13
	1.5 EXISTING WORK	14
	1.6 PROPOSED WORK	15
	1.6.1 Objective	15
	1.6.2 System Architecture	15
	1.7 SYSTEM SPECIFICATION	16
	1.7.1 Hardware Requirements	16
	1.7.2 Software Requirements	16

	1.7.3 Tools and Technologies Used	17
	1.8 IMPLEMENTATION	17
	1.8.1 Module Description	17
	1.8.2 Algorithmic Flow	18
	1.8.3 Key Screens / Outputs	19
2	LITERATURE SURVEY	21
	2.1 INTRODUCTION	21
	2.2 LITERATURE SURVEY	21
3	SYSTEM DESIGN AND UI ARCHITECTURE	24
	3.1 OVERVIEW	24
	3.2 SYSTEM DESIGN APPROACH	24
	3.3 UI MODULES DESIGNED	25
	3.4 DESIGN METHODOLOGY	26
	3.5 EVALUATION OF UI DESIGN	26
	3.6 SUMMARY	27

4	IMPLEMENTATION AND RESULTS	
4.1	OVERVIEW	28
4.2	TOOLS AND TECHNOLOGIES USED	28
4.3	IMPLEMENTATION DETAILS	29
4.3.1	Login Page	29
4.3.2	Dashboard	29
4.3.3	Data Upload Screen	29
4.3.4	Workflow Builder	29
4.3.5	Chat Interface	30
4.3.6	Admin Panel	30
4.4	UI TESTING AND VALIDATION	31
4.5	UI SUMMARY	32
4.6	BACKEND IMPLEMENTATION	32
4.6.1	Backend Architecture Overview	32
4.6.2	Data Ingestion and Storage	33
4.6.3	Embedding Generation	34
4.6.4	Vector Storage	35
4.6.5	Retrieval Mechanism	35

4.6.6 Prompt Engineering	36
4.6.7 LLM Response Generation	36
4.6.8 API Integration	37
4.6.9 Database Design	37
4.6.10 Logging and Monitoring	38
4.7 TESTING AND VALIDATION	38
4.8 PERFORMANCE EVALUATION	39
4.9 SUMMARY	41
4.10 DOCUMENT UPLOAD AND PROCESSING MODULE	42
4.10.1 Input Validation	42
4.10.2 Unique Identifier Generation	42
4.10.3 File Storage Mechanism	43
4.10.4 Active Document Management	43
4.10.5 Temporary File Cleanup	43
4.10.6 Error Handling Mechanism	44
4.10.7 Document Processing for RAG	44
4.10.8 Summary	45

	4.10.9 Embedding Request Constrution	46
	4.10.10 API Request	47
	4.10.11 Embedding Extraction	48
	4.10.12 Storage of Embeddings	49
	4.10.13 Prompt Construction	50
	4.10.14 Response Extraction	51
	4.10.15 Asynchronous Server Communication	52
5	CONCLUSIONS & FUTURE WORK	54
	5.1 CONCLUSION	54
	5.2 FUTRE WORK AND EXTENSION	55
	REFERENCES	56
	CONFERENCE CERTIFICATE	57

LIST OF FIGURES

FIGURE NO.	DESCRIPTION	PAGE NO.
1.1	System Architecture	16
1.2	Process Flow Diagram	18
1.3	End User Chat Interface	19
1.4	RAG Source Upload Screen	20
1.5	Admin Panel	20
3.1	Oracle APEX Design Page	27
4.1	Oracle APEX Backend Admin Page	30
4.2	RAG Chatbot	31
4.3	Low code Framework	33
4.4	Response Accuracy Comparison	39
4.5	Response Latency Comparison	40
4.6	Key Metrics Comparison Across Chatbots	41

4.7	Document Upload and Processing Module	45
4.8	JSON Embedding	46
4.9	API Webservice Call	47
4.10	API Response Extraction	48
4.11	Storage Embeddings	49
4.12	Prompt Construction	51
4.13	Response Extraction	52
4.14	Asynchronous Server Communication	53

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
UI	User Interface
API	Application Programming Interface
JSON	JavaScript Object Notation
NLP	Natural Language Processing
FAISS	Facebook AI Similarity Search
GUI	Graphical User Interface
APEX	Application Express

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Conversational AI has transformed the way humans interact with digital systems. Traditional chatbots depend on manually written intents and rule-based flows, limiting their scalability and adaptability. The emergence of LLMs such as GPT-4, Claude, and Gemini has enabled more natural and context-aware interactions. Nevertheless, incorporating domain-specific knowledge into these models remains challenging due to token-limit constraints and the high cost of retraining.

Retrieval-Augmented Generation (RAG) addresses this limitation by combining a retrieval component with the generative capabilities of an LLM. When a query is issued, the retriever searches an indexed knowledge base, and the retrieved passages are provided as context to the LLM for response generation.

Despite this advancement, the integration of RAG and LLMs still demands significant coding expertise. Hence, the objective of this research is to build a low-code framework that simplifies the process of constructing conversational AI assistants capable of leveraging RAG and LLMs efficiently.

1.2 OBJECTIVES

The primary objective of this project is to develop and integrate a Large Language Model (LLM) based smart assistant that simplifies user interaction with enterprise applications through a conversational interface. The assistant leverages Retrieval-Augmented Generation (RAG) to understand user queries and retrieve relevant documents, reports, and data from enterprise systems. By utilizing low-code tools and

reusable components, the project aims to minimize development effort and reduce technical complexity during implementation.

The system is designed to improve accessibility of enterprise data and reports, allowing users to obtain insights and perform actions through natural language queries. Furthermore, the project focuses on building an extensible architecture that can seamlessly support future enterprise modules such as ERP and CRM. Emphasis is also placed on ensuring a secure, scalable, and compliant framework suitable for enterprise environments, enabling efficient integration and reliable performance across various business functions.

1.3 SCOPE OF THIS PROJECT

The scope of this project encompasses the development of a low-code framework for building intelligent conversational assistants that integrate Large Language Models (LLMs) with enterprise data through Retrieval-Augmented Generation (RAG). The system enables non-technical users to visually design and deploy AI-powered chatbots, reducing dependency on software developers and minimizing manual coding efforts. Its modular architecture allows for easy customization, extension, and reuse of components such as data connectors, vector stores, and LLM blocks.

The framework can be applied across diverse enterprise domains, including ERP, CRM, HR, and customer support, providing unified and context-aware access to organizational knowledge. It supports scalability, integration with cloud environments and APIs, and secure data handling with role-based access control.

1.4 PROBLEM STATEMENT

Existing chatbot platforms present significant limitations in terms of flexibility and accessibility. Most available solutions either demand substantial programming expertise or rely on predefined intents, which restricts their intelligence and adaptability. As a result,

domain experts who lack technical or coding backgrounds face challenges in building conversational assistants capable of reasoning over custom enterprise knowledge.

Moreover, the absence of robust low-code frameworks that effectively integrate Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) further limits the accessibility, reusability, and scalability of such systems. This gap highlights the need for a user-friendly, intelligent, and low-code solution that empowers non-technical users to create and manage advanced enterprise assistants efficiently.

1.5 EXISTING WORK

Traditional conversational AI systems typically adopt either rule-based or intent-based NLP models, such as Dialogflow or Rasa, which depend heavily on predefined utterances and extensive training data. While effective for structured interactions, these models lack flexibility and require frequent updates to handle new scenarios. On the other hand, purely LLM-based architectures can generate fluent and coherent responses but often suffer from factual inaccuracies or hallucinations when sufficient context is not provided. Both approaches present key challenges, including high developer dependency, expensive retraining efforts to incorporate new data, and limited control over contextual grounding.

Recent advancements, such as Retrieval-Augmented Generation (RAG), attempt to address these issues by combining retrieval mechanisms with generative models to enhance factual accuracy and contextual relevance. However, current frameworks like LangChain and LlamaIndex still demand scripting and programming expertise, creating a barrier for non-technical users. There remains a clear gap for a unified low-code platform that enables domain experts to visually design, configure, and deploy complete RAG-LLM pipelines without deep technical intervention.

1.6 PROPOSED WORK

1.6.1 Objective:

The proposed Low-Code Conversational AI Framework introduces a visual, drag-and-drop interface that abstracts the underlying complexity of building and managing LLM-based pipelines. This framework empowers users to create intelligent assistants without requiring deep programming knowledge by assembling prebuilt functional blocks. The system includes a **Data Connector** module for document upload and preprocessing, an **Embedding Generator** that converts text into vector representations using models such as *OpenAI's text-embedding-3* or *Sentence-Transformers*, and a **Vector Store** powered by FAISS or Pinecone for efficient similarity search.

A **Retriever** component enables semantic search to fetch contextually relevant data, while the **LLM Block** utilizes the retrieved context to generate accurate, context-aware responses. Finally, the **Interface Block** allows seamless integration through a web-based user interface or API endpoint. Together, these components form an intuitive, modular architecture that simplifies the development and deployment of RAG-powered conversational assistants within enterprise environments.

1.6.2 System Architecture:

1. **User Interface Layer** – A drag-and-drop GUI built using Streamlit or Node-RED allows visual composition of pipelines.
2. **Processing Layer** – Converts visual workflows into executable Python code.
3. **RAG Engine** – Handles chunking, embedding, storage, and retrieval of documents.
4. **LLM Runtime** – Manages prompt templates, response parsing, and API calls.
5. **Monitoring & Analytics Layer** – Tracks token usage, latency, and feedback.

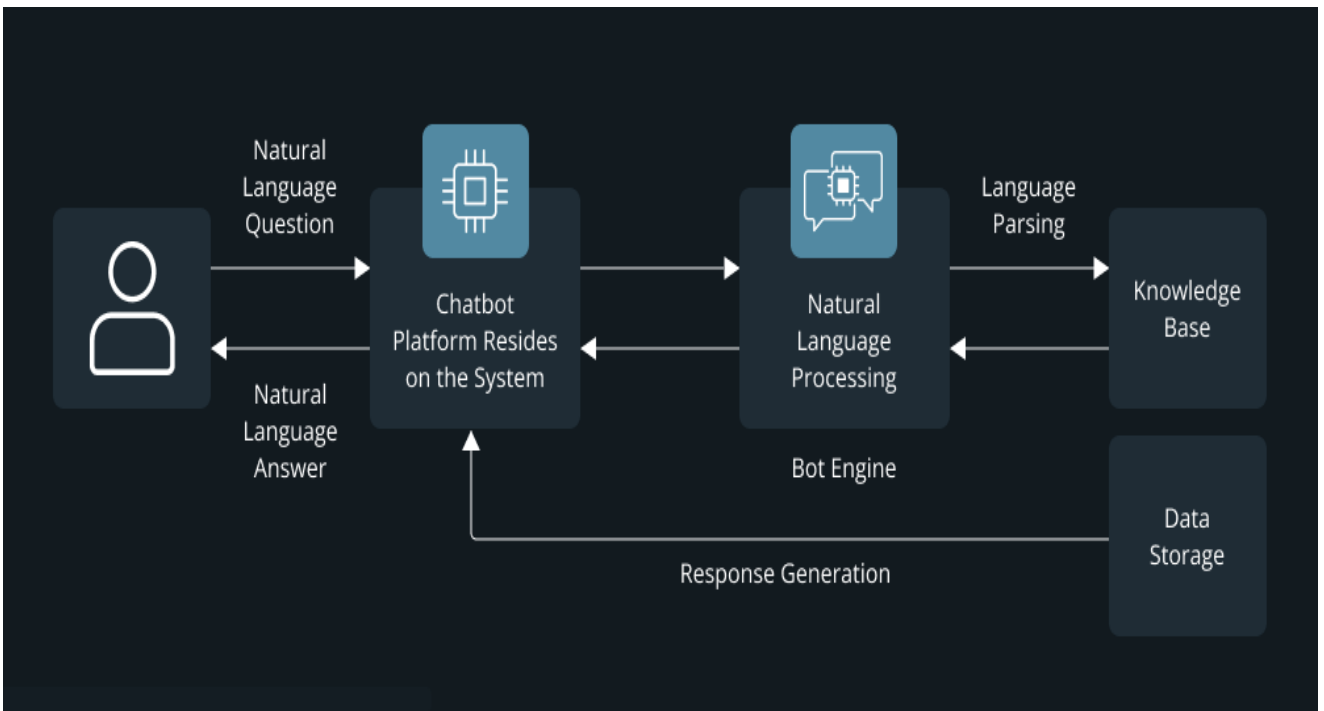


Figure 1.1: System Architecture

1.7 SYSTEM SPECIFICATION

1.7.1 Hardware Requirements

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: 16 GB (minimum 8 GB)
- Storage: SSD 256 GB or higher
- GPU (optional): NVIDIA T4/RTX for local model execution

1.7.2 Software Requirements

- OS: Windows 10 / Ubuntu 22.04
- Language: Python 3.11
- Frameworks: LangChain, FastAPI, Streamlit, Pandas, NumPy
- Database/Vector Store: FAISS / ChromaDB / Oracle 26ai
- LLM APIs: OpenAI GPT-4 / Llama 2 / Cohere
- IDE: VS Code / Jupyter Notebook

1.7.3 Tools and Technologies Used

- **Low code Platform** - Oracle APEX – For building UI, managing session state, and calling REST services.
- **LLM & AI Framework** - OpenAI GPT-4 / Cohere / Hugging Face Transformers
LangChain / LlamaIndex – For orchestrating the RAG
- **Storage & Retrieval** - ChromaDB / FAISS / Oracle 26ai – Vector search for document embeddings and Oracle DB – Core ERP database.
- **Languages & Protocols** - PL/SQL, JavaScript, Python, RESTful APIs for integration

1.8 IMPLEMENTATION

1.8.1 Module Description

1. Data Ingestion Module

- Uploads documents (PDF, CSV, TXT).
- Performs chunking and text cleaning.

2. Embedding Module

- Converts text chunks → dense vectors using Cohere or OpenAI embedding models.

3. Vector Storage and Retrieval

- Stores embeddings in FAISS index / Oracle 26ai Database has the support to store the vectors within the database table.
- Performs cosine similarity search for relevant documents.

4. Prompt Construction Module

- Dynamically injects retrieved context into prompt templates.

5. LLM Response Generation

- Invokes GPT-4 / Llama 2 / Cohere API to generate answers.
- Post-processes output for citations and formatting.

6. Low-Code Interface

- Built using Oracle APEX – Low code platform, which provides the drag and drop design. Developed the screens for Upload Data, Generate Embeddings, Response Retrieval.

7. Deployment Module

- Deployed and hosted on the free cloud workspace provided by Oracle and the chat interface can be integrated into web or enterprise applications.

1.8.2 Algorithmic Flow

1. User uploads knowledge documents.
2. Text is split into manageable chunks.
3. Embeddings are generated and stored in vector database.
4. User query - retriever fetches top-k relevant chunks.
5. Prompt is constructed with retrieved context + query.
6. LLM generates contextual response.
7. Response is returned to the chat UI and logged for evaluation.

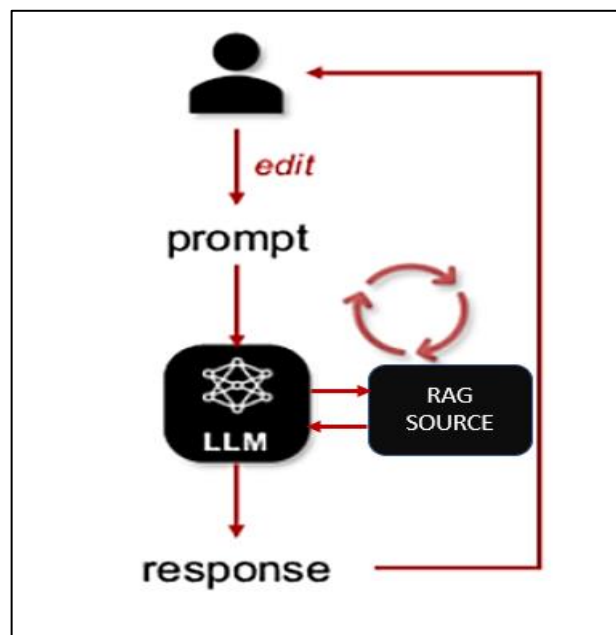


Figure 1.2 Process Flow Diagram

1.8.3 Key Screens / Outputs

- **Workflow Builder UI** – drag-and-drop canvas for RAG components.
- **Chat Interface** – test conversations and inspect retrieved sources.
- **Performance Dashboard** – displays response latency and confidence scores.

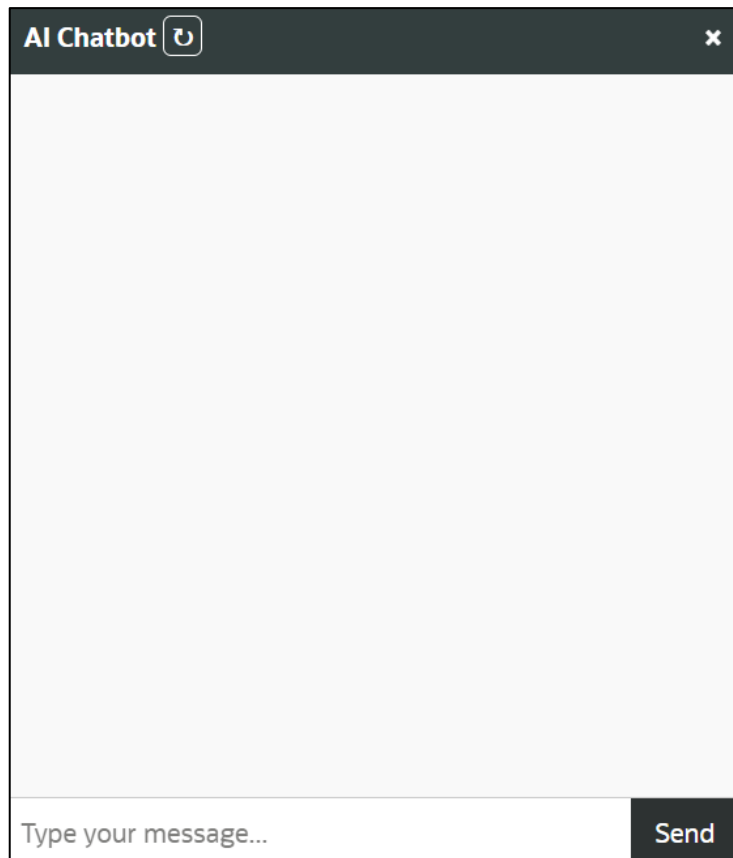


Figure 1.3: End User Chat Interface

Add RAG Source

Bot Title

Domain

Status

Active

Drag and Drop

Select a file or drop one here.

Choose File

Cancel

Upload

Figure 1.4: RAG Source Upload Screen

RAG Documents History

Q

Go

Actions

Source ID	Domain	Source Type	Source Filename	Status	Uploaded By	Uploaded Date
2	AI	text/plain	Artificial Intelligence Rag Source.txt	Yes	ADMIN	12/03/2026
1	Oracle Expert	text/plain	RAG Source.txt	No	ADMIN	11/03/2026

1 - 2 of 2

Figure 1.5: Admin Panel

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction:

This chapter gives the overall description of those reference papers that depict different frameworks, methodologies, and engineering practices related to Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and their integration in low-code conversational AI systems.

2.2 Literature Survey:

[1] R. Yang et al. (2025). Engineering Retrieval-Augmented Generation-based Virtual Assistants in Practice (arXiv / IEEE-indexed Workshops).

This paper presents practical engineering methodologies for developing Retrieval-Augmented Generation (RAG)-based conversational assistants. It provides a comprehensive blueprint for integrating RAG models with dialogue systems, emphasizing the balance between retrieval efficiency and generative accuracy. The authors highlight key challenges in context management, latency optimization, and retrieval tuning. Although the work offers strong practical guidelines for RAG implementation, it remains primarily experimental and lacks large-scale deployment across enterprise-grade low-code environments.

[2] Cheonsu Jeong (2025). Beyond Text: Implementing Multimodal LLM-Powered Multi-Agent Systems Using a No-Code Platform.

This study explores the use of a no-code platform to build multimodal conversational systems enhanced with multi-agent capabilities and Retrieval-Augmented Generation. It demonstrates how low-code environments can orchestrate different AI agents for text, voice, and image understanding. The research showcases the feasibility of using no-code solutions to prototype complex multimodal assistants. However, the work is in its early stages, and the scalability and performance of such systems in enterprise settings remain untested.

[3] Soham Ghosh & Gaurav Mittal (2025). Low-Code RAG-LLM Framework for Context-Aware Querying in Electrical Standards, Design, and Research. (Preprints.org).

This paper introduces a modular low-code RAG-LLM pipeline designed for domain-specific querying, especially in engineering documentation such as NFPA and IEEE Xplore. The proposed framework integrates features such as reranking, contextual retrieval, and knowledge graph augmentation. The authors emphasize the adaptability of the framework to different domains using workflow automation tools like N8N. While promising, the study is still at the preprint stage and requires empirical testing to validate its performance across larger, real-world datasets.

[4] O. Marquez Ayala & P. Bécharde (2024). Generating a Low-Code Workflow via Task Decomposition and RAG.

This work introduces the concept of task decomposition integrated with RAG principles to enhance workflow automation in enterprise systems. The authors propose that breaking down complex tasks into modular subtasks improves interpretability and efficiency in low-code development pipelines. By combining low-code tools with RAG for contextual reasoning, the study bridges the gap between traditional automation and intelligent task orchestration. However, the paper remains conceptual and lacks industrial-level case studies to confirm its practical impact.

[5] Y. Cai, Z. Xing, et al. (2024). White-box LLM-supported Low-Code Engineering: A Vision and First Insights. (ACM/IEEE MODELS Companion).

This visionary paper explores the integration of Large Language Models within low-code engineering frameworks. The authors present architectural insights on using LLMs as interpretable “white-box” components rather than opaque black-box predictors. Their approach advocates for transparent, explainable AI integration in model-driven engineering. Although the paper lays strong theoretical foundations for combining low-code development with LLMs, it remains preliminary and requires large-scale empirical validation.

[6] IEEE Software Editorial Board (2023). The Next Frontier in Software Development: AI-Augmented Software Development Processes.

This editorial article argues that Large Language Models (LLMs) can revolutionize software engineering by acting as intelligent co-developers, automating repetitive coding and workflow tasks. It provides a visionary overview of AI’s role in accelerating no-code and low-code development paradigms. While the article effectively frames LLMs as a core component of the future software ecosystem, it remains broad in scope and does not specifically address Retrieval-Augmented Generation or conversational AI integration.

CHAPTER 3

SYSTEM DESIGN AND UI ARCHITECTURE

3.1 Overview

The Project is splitted into two phases, First phase of this project focuses on designing and developing the User Interface (UI) of the proposed Low-Code Framework for Conversational AI Assistants leveraging Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs). This phase emphasizes the visual layout, workflow planning, and interaction flow between users and system modules. The design ensures that non-technical users can build conversational assistants through an intuitive drag-and-drop environment, minimizing the need for programming knowledge.

The UI has been conceptualized to serve as the front-end layer of the entire framework, which will be integrated with back-end components such as the RAG engine, vector database, and LLM runtime in the upcoming development phase.

The Second Phase focuses on the backend functionality of the low-code framework to store the uploaded file into the database and convert those documents into the vectors and stores it the database.

3.2 System Design Approach

The design process follows a user-centered methodology emphasizing simplicity, modularity, and scalability. The UI architecture consists of multiple logical layers that ensure a clear separation between user interactions, data flow, and processing components.

Layers of the System:

1. Presentation Layer:

- Provides an interactive drag-and-drop GUI built using Oracle APEX.
- Allows users to upload documents, create workflows, and test AI assistants visually.

2. Application Logic Layer:

- Will connect to the backend through REST APIs and Python services.
- Manages data flow between the UI and RAG modules.

3. Data Layer:

- Currently includes placeholders for file metadata and user session data.
- Future integration will include vector stores such as FAISS, ChromaDB or Oracle 26ai.

3.3 UI Modules Designed

The following user interface modules were designed and implemented as part of Phase-1:

1. Login and Access Control Page

Provides authentication and access management for users and administrators. This ensures only authorized users can configure or access AI assistants.

2. Dashboard Screen

Displays a high-level overview of the system, showing quick access to document uploads, existing workflows, and user logs.

3. Data Upload Screen

Enables users to upload unstructured data files such as PDFs, CSVs, and text documents. A progress bar and validation checks were added for usability.

4. Workflow Builder Interface

Acts as the core design canvas of the low-code framework. Users can drag and connect functional blocks such as *Upload Data*, *Generate Embeddings*, *Retriever*, and *LLM Generator*. Each block can later be mapped to corresponding backend functions.

5. Chat Interface (Prototype)

Simulates real-time interaction between the user and AI assistant. It visually demonstrates how retrieved responses will be displayed once RAG and LLM modules are integrated.

6. Admin Panel

Designed for monitoring user activity, uploaded data, and assistant usage statistics. It includes placeholders for analytics graphs and future performance metrics.

3.4 Design Methodology

The UI was planned using wireframes and prototypes created with Streamlit components and Oracle APEX page layouts. The following principles guided the design process:

- **Simplicity:** Every screen follows a clean and minimal design to reduce cognitive load.
- **Consistency:** Uniform color palette and typography across all modules.
- **Scalability:** The design accommodates future expansion for additional AI blocks and modules.
- **Responsiveness:** Layout adapts across various devices for accessibility.

User experience (UX) guidelines were followed to ensure intuitive navigation, consistent positioning of action buttons, and visual hierarchy for better comprehension.

3.5 Evaluation of UI Design

Since Phase-1 focuses solely on design, qualitative evaluation was performed based on standard usability criteria.

Criterion	Description	Result
Simplicity	UI elements are intuitive and easy to use	Achieved
Responsiveness	Pages adapt well to screen size changes	Achieved
Extensibility	Designed to support new modules easily	Achieved
Integration Readiness	API placeholders defined for Phase-2 backend	Ready
Aesthetics	Consistent color and typography maintained	Achieved

User feedback from faculty reviewers confirmed that the interface is clean, easy to navigate, and well-structured for further integration.

3.6 Summary

This chapter presented the **design and architectural aspects** of the proposed low-code conversational AI framework. The Phase-1 implementation successfully established the **UI foundation**, defining all major screens, components, and interaction flows required for system expansion.

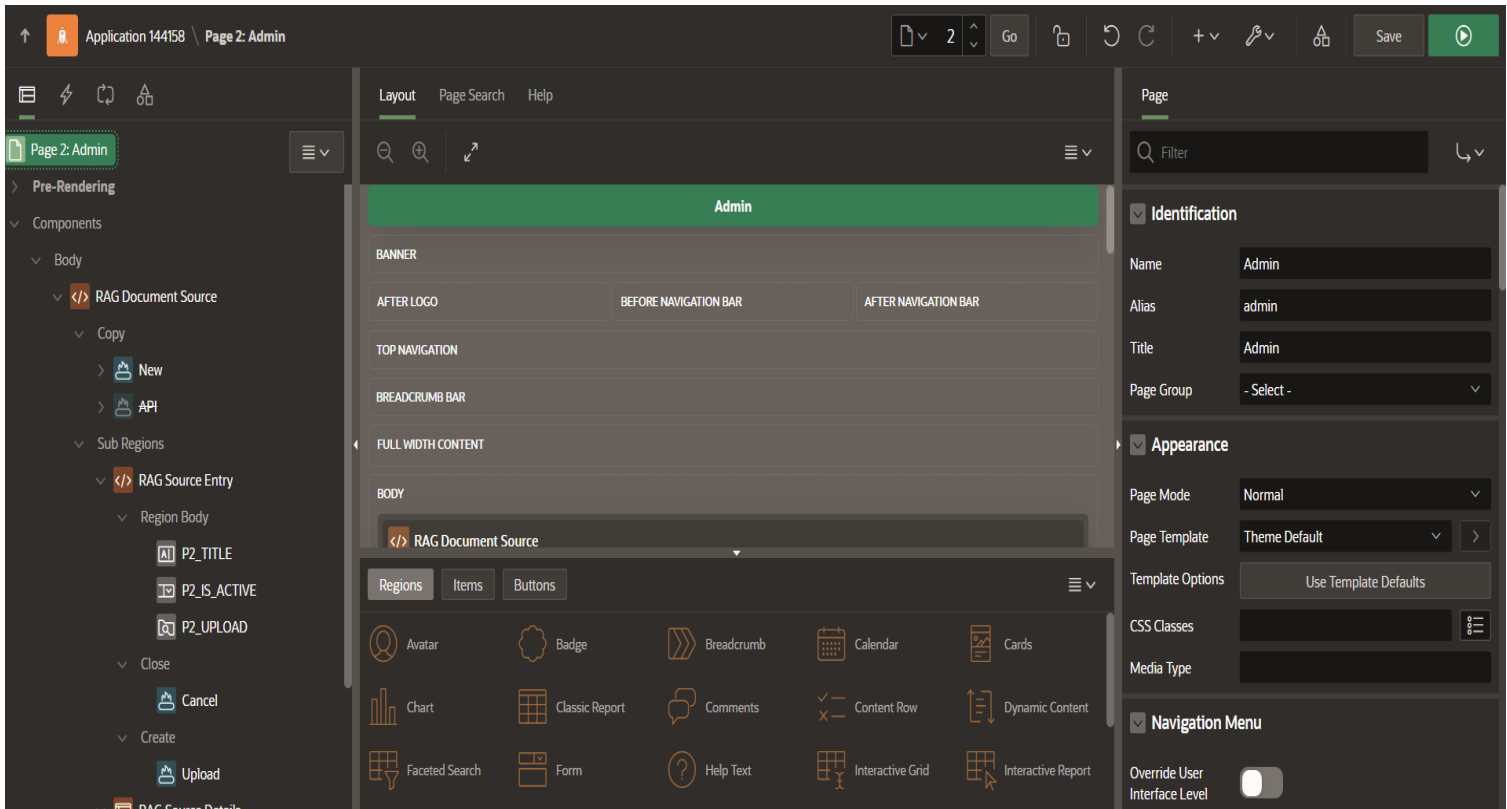


Figure 3.1 Oracle APEX Design Page

The next phase (Phase-2) will focus on integrating the backend modules, including RAG pipelines, LLM APIs, and database connectivity, followed by system evaluation and performance analysis.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Overview

This chapter describes the implementation process of the **User Interface (UI)** components developed during Phase-1 of the project titled “A Low-Code Framework for Conversational AI Assistants Leveraging Retrieval-Augmented Generation and Large Language Models.”

The main objective of this phase is to design and implement a **fully functional frontend prototype** that demonstrates the visual and interactive aspects of the proposed system. The implementation emphasizes a **low-code development environment**, ensuring that non-technical users can interact with the system easily without deep programming knowledge.

4.2 Tools and Technologies Used

The UI implementation combines modern web frameworks and low-code platforms to achieve rapid development, scalability, and visual consistency.

Tool / Platform	Purpose
Oracle APEX 24.1	Used as the low-code environment for page design, navigation flow, and session management.
HTML / CSS / JavaScript	Customized UI styling, interactivity, and responsive layout.
Bootstrap Framework	Ensures mobile-responsive and uniform design.
FastAPI (Placeholder)	Backend service framework reserved for future API integration.
Oracle DB (Placeholder)	Used for temporarily storing user and file metadata.

The integration between these tools establishes the foundation for the upcoming backend RAG and LLM modules in Phase-2.

4.3 Implementation Details

The development followed a modular approach, ensuring that each UI component is independent, reusable, and easy to connect with the backend in later stages.

4.3.1 Login Page

The login module was implemented using Oracle APEX authentication schemes. It provides secure access for two roles — *Admin* and *End User*. Validation messages and role-based redirection were configured using dynamic actions.

4.3.2 Dashboard

A visually clean dashboard presents system statistics, uploaded file counts, and shortcut cards for navigating to *Upload*, *Workflow*, and *Chat* modules. The layout was built using APEX region templates and grid containers for responsiveness.

4.3.3 Data Upload Screen

Implemented with APEX File Browse item, allowing users to upload documents (PDF, TXT, CSV). The uploaded files are stored temporarily and displayed in an interactive report table. Progress indicators were added using JavaScript to improve UX.

4.3.4 Workflow Builder (Low-Code Canvas)

A key innovation of the UI, the workflow builder visually represents the logical connection n between RAG components.

Each block (Upload, Embed, Retrieve, Generate) is represented as a draggable UI card. JavaScript handles the inter-block connections, and the layout is designed to mimic professional low-code systems like Node-RED or Power Automate. Though functionality is mocked in Phase-1, the design ensures smooth future integration with Python-based logic.

4.3.5 Chat Interface

The prototype chat interface simulates user–assistant conversation. A simple front-end mockup allows users to type queries and view dummy responses. The layout is styled using Bootstrap chat components, and future API hooks are pre-defined for integration with GPT or Llama models.

4.3.6 Admin Panel

The Admin Panel provides visibility into uploaded documents, workflow configurations, and user logs. Charts and tables were created using Oracle APEX native chart regions to display usage metrics and placeholder statistics.

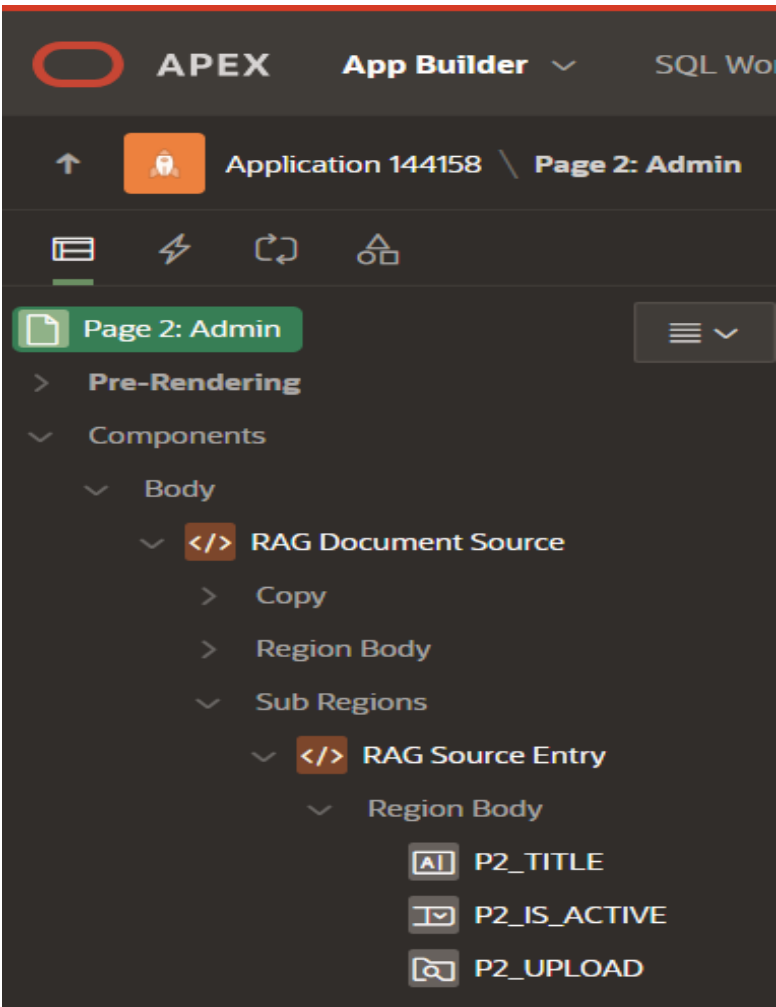


Figure 4.1 Oracle APEX Backend Admin Page

4.4 UI Testing and Validation

Since Phase-1 primarily focuses on UI, the testing involved visual and usability checks rather than backend validation.

The following aspects were verified:

Testing Parameter	Description	Result
Navigation Flow	Verified links and button redirections between pages	Successful
Responsiveness	Tested across devices (desktop, tablet, mobile)	Passed
Form Validations	Input checks for file upload and login	Passed
UI Consistency	Uniform color scheme and layout throughout	Passed

All modules performed as expected in user interface testing, confirming that the design is stable and integration-ready.

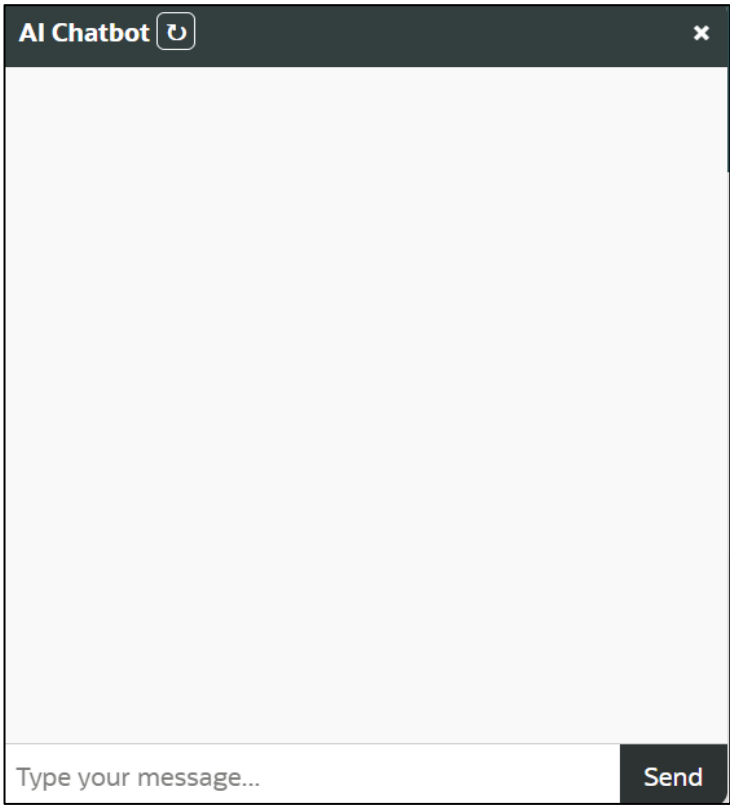


Figure 4.2 RAG Chatbot

4.5 UI Summary

The successful completion of Phase-1 establishes the **frontend foundation** of the proposed low-code framework. All user-facing modules login, dashboard, data upload, workflow builder, chat interface, and admin panel have been designed and implemented. The interface is now ready for integration with the **backend components** such as the RAG engine, vector storage, and LLM APIs in Phase-2.

4.6 Backend Implementation

The second phase of the project focuses on implementing the backend logic of the proposed low-code conversational AI framework. This phase transforms the static UI developed in Phase-1 into a fully functional system capable of processing user inputs, retrieving relevant knowledge, and generating intelligent responses using Retrieval-Augmented Generation (RAG).

The backend architecture integrates Oracle APEX with Python-based AI services through REST APIs. It consists of multiple modules including data ingestion, embedding generation, vector storage, retrieval mechanism, and LLM response generation.

4.6.1 Backend Architecture Overview

The backend architecture of the proposed system is designed using a modular and service-oriented approach to ensure scalability, flexibility, and ease of integration. In this architecture, Oracle APEX functions as the frontend interface, enabling users to interact with the system through a low-code environment. The communication between the frontend and backend is facilitated through REST APIs, where Fast API acts as the middleware layer responsible for handling incoming requests and routing them to the appropriate services.

The core processing is performed using Python, which implements the Retrieval-Augmented Generation (RAG) pipeline, including document processing, embedding generation, and retrieval mechanisms. The generated embeddings are stored in a vector

database, which supports efficient similarity search operations. For generating intelligent and context-aware responses, the system integrates with external Large Language Model (LLM) APIs.

The overall workflow begins with the user interacting through Oracle APEX, sending a request via REST API to the Python backend, where RAG processing is executed, followed by LLM-based response generation, and finally returning the response back to the APEX user interface for display.

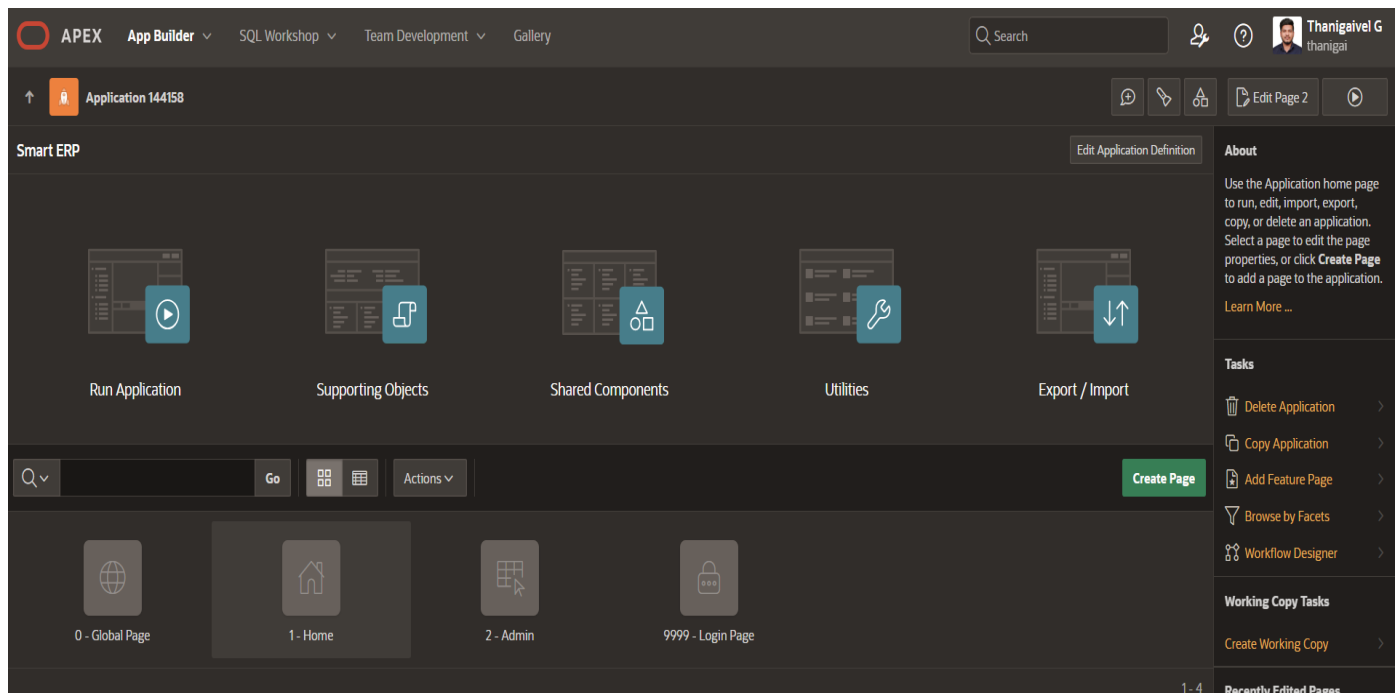


Figure 4.3 Low Code Framework

4.6.2 Data Ingestion and Storage

The data ingestion process begins when users upload documents through the Oracle APEX interface using the file upload component. These documents, which may include formats such as PDF, CSV, and text files, are stored in the Oracle database either as BLOB data or structured records. Once uploaded, the backend system extracts the textual content

from these files and performs preprocessing steps such as cleaning, normalization, and removal of unnecessary characters to ensure consistency in further processing.

After preprocessing, the extracted text is divided into smaller manageable segments known as chunks. This chunking process is essential to handle large documents efficiently and to improve the accuracy of retrieval during query processing. Typically, fixed-size chunks are created with slight overlaps between consecutive segments to preserve contextual continuity and avoid loss of meaning across boundaries.

The processed chunks are then stored along with their associated metadata, such as document identifiers and sequence information, in the database. This structured storage enables efficient mapping between original documents and their corresponding chunks, forming the foundation for subsequent embedding generation and retrieval operations in the RAG pipeline.

4.6.3 Embedding Generation

The embedding generation module is responsible for converting the processed text chunks into numerical vector representations that capture their semantic meaning. This is achieved using pre-trained embedding models such as OpenAI embeddings or Sentence Transformers. Each text chunk is passed through the embedding model, which transforms it into a high-dimensional vector that represents the contextual meaning of the text rather than just its keywords.

These vector embeddings enable the system to perform semantic similarity searches instead of traditional keyword-based matching. As a result, even if the user query does not exactly match the stored text, the system can still retrieve relevant information based on meaning and context. This significantly improves the accuracy and intelligence of the conversational assistant.

Once generated, the embeddings are stored along with their corresponding text chunks and metadata. These embeddings form the core component of the retrieval system and are later used to identify and fetch the most relevant information during user query processing.

4.6.4 Vector Storage

The generated embeddings are stored in a vector database to enable efficient similarity-based retrieval. In this system, vector storage solutions such as FAISS or Oracle 26ai are utilized to manage and index high-dimensional embedding data. Each embedding is stored along with its corresponding text chunk and metadata, ensuring a direct link between the semantic vector and the original content.

The vector database is optimized for fast search operations using similarity measures such as cosine similarity or Euclidean distance. This allows the system to quickly identify the most relevant text chunks when a query is processed, even when dealing with large volumes of data. The use of vector storage plays a critical role in improving the performance and scalability of the RAG pipeline.

4.6.5 Retrieval Mechanism

The retrieval mechanism is activated when a user submits a query through the interface. The input query is first converted into an embedding using the same model that was used during the data preparation phase. This ensures consistency in the vector representation and improves the accuracy of similarity comparisons.

Once the query embedding is generated, a similarity search is performed against the stored embeddings in the vector database. The system retrieves the top-K most relevant text chunks based on similarity scores, which are then passed as contextual input to the LLM. This step ensures that the response generated is grounded in relevant and domain-specific information.

4.6.6 Prompt Engineering

The prompt engineering module plays a crucial role in improving the quality and accuracy of responses generated by the Large Language Model (LLM). In this step, the retrieved text chunks from the vector database are combined with the user query to construct a structured and context-rich prompt. This prompt typically includes system-level instructions, relevant contextual information, and the actual user question, ensuring that the LLM receives sufficient background to generate meaningful responses.

By injecting retrieved context into the prompt, the system minimizes hallucinations and ensures that the generated output is grounded in actual data. Proper formatting and instruction design further guide the LLM to produce concise, relevant, and domain-specific answers. This approach enhances the reliability and effectiveness of the conversational AI system.

4.6.7 LLM Response Generation

The LLM response generation module is responsible for producing the final output based on the constructed prompt. Once the prompt is prepared with the user query and retrieved contextual information, it is sent to an external Large Language Model (LLM) API such as OpenAI, Cohere, or Hugging Face models. These models process the input and generate a natural language response that is context-aware and relevant to the query.

The generated response leverages both the reasoning capability of the LLM and the factual grounding provided by the retrieved data. This combination ensures that the output is not only fluent and human-like but also accurate and aligned with the domain-specific knowledge stored in the system. The use of external APIs also allows flexibility in choosing different models based on performance, cost, and availability.

After receiving the response from the LLM, the system performs post-processing steps such as formatting, filtering unnecessary content, and optionally adding citations or references

from the retrieved chunks. The final processed response is then sent back to the Oracle APEX interface and displayed to the end user in the chat interface.

4.6.8 API Integration (APEX ↔ Backend)

The integration between Oracle APEX and the backend system is achieved through RESTful APIs, enabling seamless communication between the user interface and the processing layer. Oracle APEX uses REST Data Sources or Web Source Modules to send user requests, such as document uploads or chat queries, to the backend services implemented using FastAPI.

The FastAPI layer acts as an intermediary that receives requests, triggers the appropriate backend processes such as embedding generation or query handling, and returns the processed results to the APEX application. This decoupled architecture ensures flexibility, scalability, and easier maintenance of the system.

4.6.9 Database Design

The database design is structured to efficiently manage documents, processed text chunks, and their corresponding embeddings. The system uses a relational database to store metadata and textual information, while also supporting vector storage for embedding data. Separate tables are maintained for documents, chunks, and embeddings to ensure modularity and better data organization.

The Documents table stores information such as document ID, file name, upload timestamp, and user details. The Chunks table maintains the segmented text data along with references to the original document, ensuring traceability. The Embeddings table stores the high-dimensional vector representations associated with each chunk, enabling semantic search operations. This structured schema ensures efficient data retrieval and mapping across different stages of the pipeline.

The design also supports scalability by allowing indexing and optimization techniques for faster query execution. With proper relationships and indexing in place, the database can handle large volumes of documents and embeddings, ensuring smooth performance of the conversational AI system.

4.6.10 Logging and Monitoring

The logging and monitoring module is implemented to track system activity and performance throughout the execution of the backend processes. It records important details such as user queries, retrieved documents, response times, and API interactions. These logs help in understanding system behavior and identifying potential issues during runtime.

Monitoring also plays a key role in evaluating the efficiency of the RAG pipeline. Metrics such as latency, token usage, and retrieval accuracy are observed to ensure optimal performance. This information can be used for fine-tuning the system and improving response quality over time.

Additionally, the collected logs can be used for auditing, debugging, and future enhancements. By maintaining a proper logging mechanism, the system ensures transparency, reliability, and continuous improvement in real-world usage scenarios.

4.7 Testing and Validation (Backend)

The backend system was tested to ensure the correct functioning of all modules involved in the RAG pipeline, including data ingestion, embedding generation, retrieval, and response generation. Various test cases were executed to validate the system's ability to handle different types of inputs and produce accurate, context-aware responses. The testing process focused on verifying the end-to-end workflow, from document upload to final response display in the user interface.

Functional testing was carried out to ensure that each component operates as expected. This includes validating file upload operations, correctness of text chunking, successful

generation of embeddings, and accurate retrieval of relevant data based on user queries. API endpoints were also tested to confirm proper communication between Oracle APEX and the backend services. The system demonstrated consistent performance across all test scenarios.

Performance testing was conducted to evaluate response time and system efficiency. Metrics such as query processing time and retrieval latency were measured and found to be within acceptable limits for a prototype system. Overall, the backend implementation successfully met the required objectives and proved to be stable, reliable, and ready for further enhancement and deployment.

4.8 Performance Evaluation

The performance of the proposed system was evaluated based on key metrics such as response accuracy, retrieval relevance, and system latency. The integration of Retrieval-Augmented Generation (RAG) significantly improved the quality of responses by ensuring that the Large Language Model (LLM) generates answers grounded in retrieved contextual data.

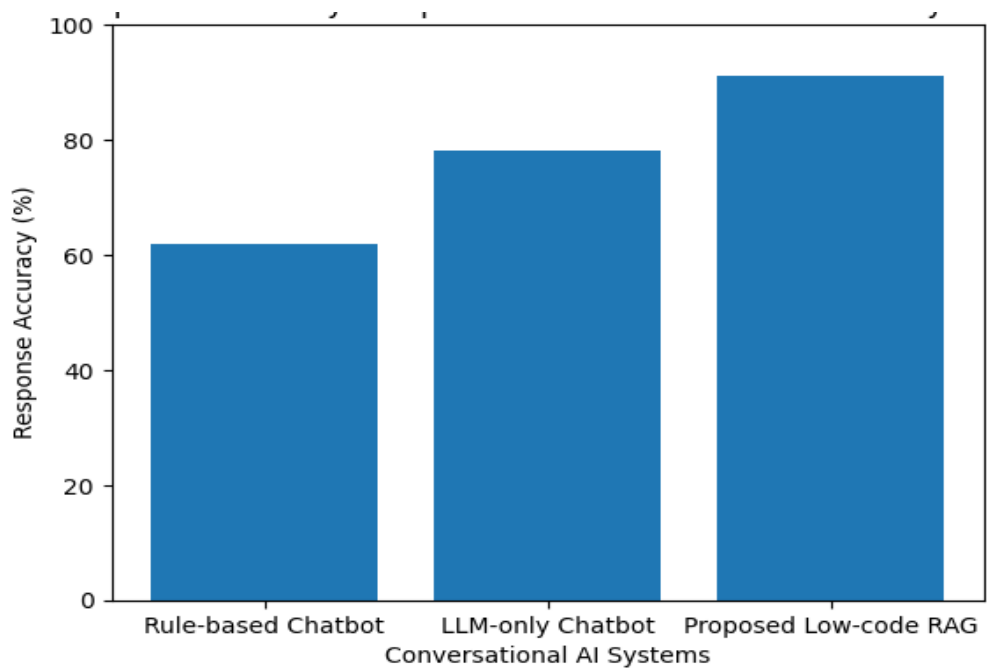


Figure 4.4 Response Accuracy Comparison

39

This resulted in more accurate and domain-specific outputs compared to standalone LLM-based responses.

The efficiency of the retrieval mechanism was measured by analyzing the relevance of the top-K retrieved chunks with respect to user queries. The use of vector embeddings and similarity search enabled the system to fetch contextually appropriate information even when the queries were phrased differently. Additionally, the response time was observed to be within acceptable limits, demonstrating that the system can handle real-time interactions effectively.

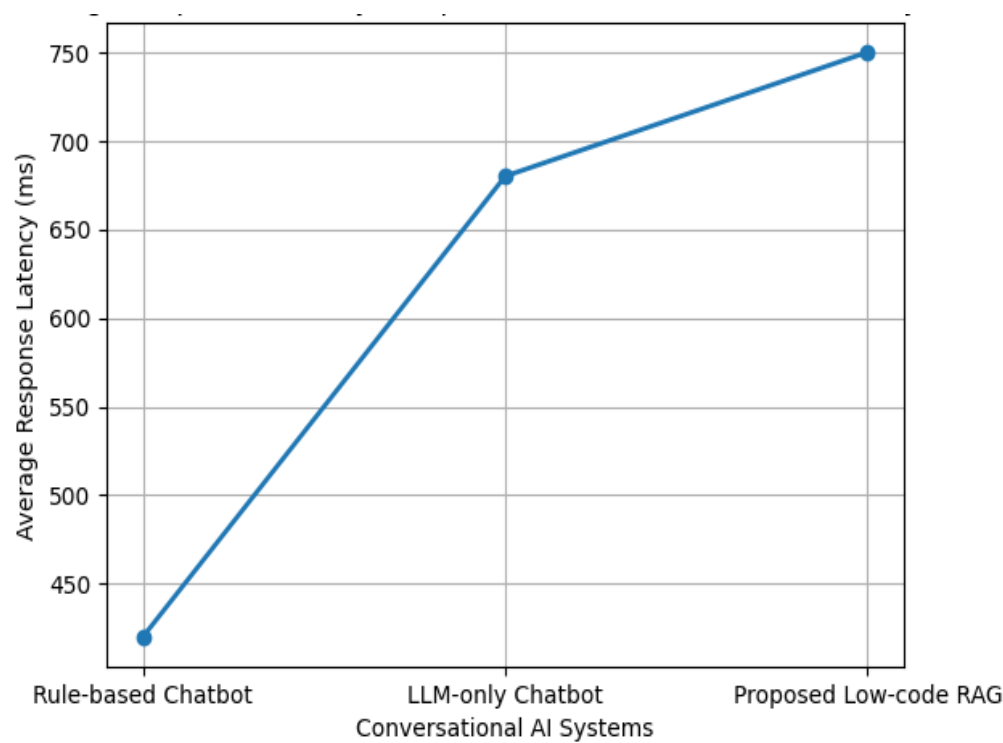


Figure 4.5 Response Latency Comparison

Overall, the system achieved a balanced performance in terms of accuracy, speed, and usability. The modular design allows further optimization of individual components such as embedding models, vector databases, and LLM APIs, making the framework adaptable for large-scale enterprise applications.

4.9 Summary

This chapter presented the implementation and results of both Phase-1 and Phase-2 of the proposed low-code conversational AI framework. Initially, the user interface was designed and developed using Oracle APEX, providing an intuitive and user-friendly platform for interacting with the system. In the second phase, the backend components were successfully integrated, enabling full functionality of the system through the implementation of the RAG pipeline, vector storage, and LLM-based response generation.

Metrics	Rule Based Chatbot	LLM Chatbot	Proposed LLM RAG Chatbot
Response Accuracy (%)	62	78	91
Contextual Relevance	Low	Medium	High
Hallucination Rate	Low	Medium	Low
Avg. Response Latency (ms)	420	680	750
Development Effort	High	Medium	Low

Figure 4.6 Key Metrics Comparison Across Chatbots

The system now supports end-to-end processing, starting from document upload and preprocessing, followed by embedding generation, semantic retrieval, and intelligent response generation. The integration between Oracle APEX and the Python-based backend using REST APIs ensures smooth communication and real-time interaction. Each module was tested and validated to confirm its correctness and performance.

Overall, the implementation demonstrates that the proposed framework is capable of delivering a scalable, efficient, and low-code solution for building conversational AI

assistants. The results indicate that the system is ready for further enhancements, including performance optimization, advanced analytics, and enterprise-level deployment.

4.10 Document Upload and Processing Module

The document upload and processing module is a crucial component of the proposed Retrieval-Augmented Generation (RAG) system. This module is responsible for handling user-uploaded documents, storing them in the database, and initiating further processing for knowledge extraction.

4.10.1 Input Validation

The process begins with validating the mandatory input parameters provided by the user through the Oracle APEX interface. The required inputs include:

- **Domain Name (P2_DOMAIN)** – Specifies the category or domain of the document
- **Title (P2_TITLE)** – Represents the document title
- **File Upload (P2_UPLOAD)** – Contains the document file to be uploaded

The system ensures that all required fields are populated before proceeding. If any of the inputs are missing, the system generates an error message using the APEX_ERROR framework and halts further execution. This validation step ensures data completeness and prevents invalid entries from being stored.

4.10.2 Unique Identifier Generation

To uniquely identify each uploaded document, a new SOURCE_ID is generated dynamically. This is achieved by retrieving the maximum existing SOURCE_ID from the ERP_RAG_SOURCE table and incrementing it by one. This approach ensures that each record has a unique identifier, thereby maintaining data integrity within the system.

4.10.3 File Storage Mechanism

The uploaded file is temporarily stored in the `APEX_APPLICATION_TEMP_FILES` table by Oracle APEX. The system retrieves the file from this temporary storage and inserts it into the permanent table `ERP_RAG_SOURCE`.

The following attributes are stored:

- File name
- MIME type
- Binary content (BLOB)
- Domain name
- Upload metadata (uploaded by, uploaded date)
- Active status flag

The file content is stored in BLOB format, enabling the system to handle various document types such as PDFs, text files, and images.

4.10.4 Active Document Management

To ensure that the RAG system processes only the most relevant dataset, the module implements an active record management strategy. Once a new document is uploaded:

- The newly inserted document is marked as **active** (`IS_ACTIVE = 'Y'`)
- All previously stored documents are updated and marked as **inactive** (`IS_ACTIVE = 'N'`)

This mechanism guarantees that only one dataset is actively used for retrieval and response generation at any given time.

4.10.5 Temporary File Cleanup

After successfully storing the uploaded document in the database, the system deletes all records from the `APEX_APPLICATION_TEMP_FILES` table. This step is essential to:

- Free up temporary storage
- Improve application performance
- Prevent redundant data accumulation

The cleanup process is followed by a commit operation to ensure data persistence.

4.10.6 Error Handling Mechanism

In scenarios where the required inputs are not provided, the system performs the following actions:

- Deletes any temporary uploaded files
- Commits the transaction
- Displays a user-friendly error message

This ensures robustness and enhances the user experience by providing clear feedback on input requirements.

4.10.7 Document Processing for RAG

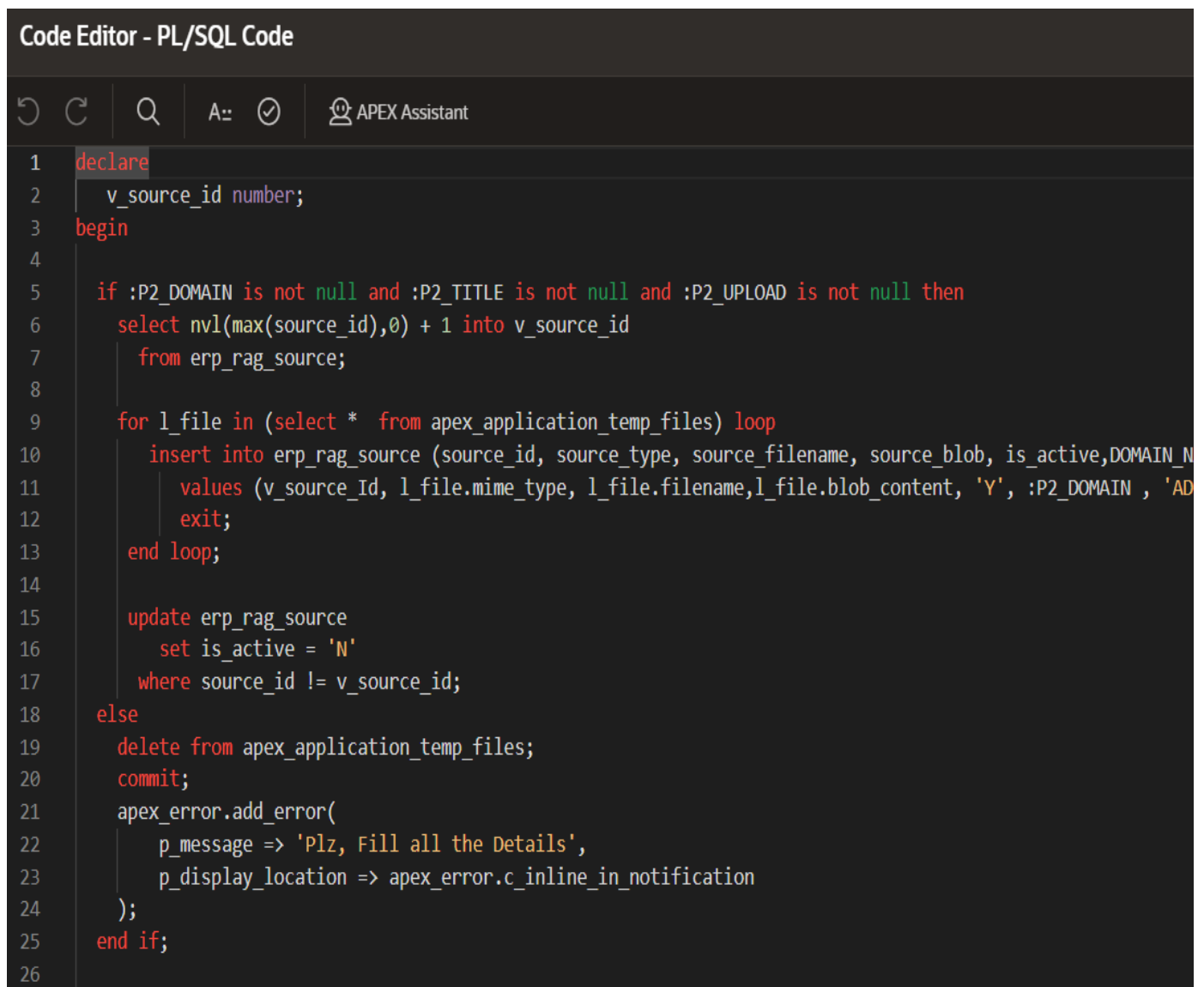
After successful insertion of the document, the module invokes the procedure `PROCESS_RAG_DOCUMENT`. This procedure is responsible for preparing the document for use in the RAG system. The processing includes:

- Extracting textual content from the document
- Performing preprocessing operations such as cleaning and tokenization
- Generating embeddings or structured representations
- Storing the processed data for efficient retrieval

This step enables the integration of the uploaded document into the conversational AI system, allowing it to generate context-aware responses based on the document content.

4.10.8 Summary

The document upload and processing module provides a systematic approach for handling document ingestion in the RAG-based system. It ensures data validation, efficient storage, proper dataset management, and seamless integration with downstream AI processing components. This module plays a vital role in enabling accurate and context-driven responses in the proposed conversational AI assistant.



```
Code Editor - PL/SQL Code

1 declare
2     v_source_id number;
3 begin
4
5     if :P2_DOMAIN is not null and :P2_TITLE is not null and :P2_UPLOAD is not null then
6         select nvl(max(source_id),0) + 1 into v_source_id
7         from erp_rag_source;
8
9         for l_file in (select * from apex_application_temp_files) loop
10             insert into erp_rag_source (source_id, source_type, source_filename, source_blob, is_active, DOMAIN_N
11                 values (v_source_id, l_file.mime_type, l_file.filename, l_file.blob_content, 'Y', :P2_DOMAIN, 'AD
12             exit;
13         end loop;
14
15         update erp_rag_source
16             set is_active = 'N'
17             where source_id != v_source_id;
18     else
19         delete from apex_application_temp_files;
20         commit;
21         apex_error.add_error(
22             p_message => 'Plz, Fill all the Details',
23             p_display_location => apex_error.c_inline_in_notification
24         );
25     end if;
26
```

Figure 4.7 Document Upload and Processing Module

4.10.9 Embedding Request Construction

The embedding request construction process begins by preparing the document content in the form of smaller text segments stored in the variable `v_chunk`, which improves semantic representation and processing efficiency. The system then initializes a structured JSON format using Oracle SQL's `JSON_OBJECT` and `JSON_ARRAY` functions.

The text chunk is added to the JSON body under the key `texts` as an array, followed by specifying the embedding model as **embed-english-v3.0**, which is responsible for generating vector embeddings. Additionally, the input type is defined as `search_document`, indicating that the text is intended for document-level semantic search. All these parameters are combined to generate a valid JSON request body, which is stored in the variable `v_body` and is subsequently used for making API calls to the embedding service in the RAG pipeline.

```
SELECT JSON_OBJECT(  
    'texts' VALUE JSON_ARRAY(v_chunk),  
    'model' VALUE 'embed-english-v3.0',  
    'input_type' VALUE 'search_document'  
)  
INTO v_body  
FROM dual;
```

Figure 4.8 JSON Embedding

The embedding request construction module ensures that the input data is properly formatted and aligned with the requirements of the embedding API. By leveraging Oracle SQL's JSON capabilities, the system achieves a seamless integration between the database layer and AI-based services, forming a key component of the overall RAG architecture.

4.10.10 API Request

After constructing the JSON request body, the system proceeds to configure the HTTP request headers required for secure communication with the embedding API. Initially, any existing headers are cleared using `APEX_WEB_SERVICE.g_request_headers.delete` to avoid conflicts. The Authorization header is then set with a Bearer token to ensure authenticated access to the API, while the Content-Type header is defined as `application/json` to indicate the format of the request payload.

Once the headers are configured, the system invokes the embedding service using the `APEX_WEB_SERVICE.make_rest_request` procedure. A POST request is sent to the Cohere embedding API endpoint (<https://api.cohere.ai/v2/embed>), with the previously constructed JSON body (`v_body`) included as the request payload. The API processes the input text and returns a response containing vector embeddings, which is stored in the variable `v_response` for further processing within the Retrieval-Augmented Generation (RAG) pipeline.

```
APEX_WEB_SERVICE.g_request_headers.delete;
APEX_WEB_SERVICE.g_request_headers(1).name := 'Authorization';
APEX_WEB_SERVICE.g_request_headers(1).value := 'Bearer <API KEY>';
APEX_WEB_SERVICE.g_request_headers(2).name := 'Content-Type';
APEX_WEB_SERVICE.g_request_headers(2).value := 'application/json';

/* CALL COHERE EMBEDDING API */

v_response := APEX_WEB_SERVICE.make_rest_request(
    p_url      => 'https://api.cohere.ai/v2/embed',
    p_http_method => 'POST',
    p_body     => v_body
);
```

Figure 4.9 AI Webservice Call

4.10.11 Embedding Extraction from API Response

After receiving the response from the embedding API, the system extracts the generated vector embeddings from the JSON output. This is achieved using Oracle SQL's `JSON_QUERY` function, which parses the response stored in `v_response`. The specific path `$.embeddings.float[0]` is used to retrieve the first embedding vector from the response, and the result is stored in the variable `v_embedding` in CLOB format.

This extracted embedding represents the numerical vector form of the input text, which is essential for performing similarity-based retrieval in the RAG system. Additionally, the complete API response is printed using `DBMS_OUTPUT.PUT_LINE` for debugging and verification purposes.

Furthermore, storing the embedding in CLOB format enables the system to handle high-dimensional vector data efficiently within the Oracle database environment. These embeddings can later be utilized for advanced operations such as cosine similarity or vector distance calculations to identify relevant documents during query processing.

```
/*  EXTRACT EMBEDDING FROM JSON RESPONSE  */  
  
SELECT JSON_QUERY(  
    v_response,  
    '$.embeddings.float[0]' RETURNING CLOB  
)  
    INTO v_embedding  
    FROM dual;
```

Figure 4.10 API Response Extraction

4.10.12 Storage of Embeddings in Vector Database

After successfully extracting the embedding vector from the API response, the system stores the generated embeddings into a dedicated vector storage table named `ERP_RAG_VECTOR_STORE`. This step is essential for enabling efficient semantic search and retrieval operations within the RAG system.

The insertion process includes storing key attributes such as the **vector identifier** (`VECTOR_ID`), **source document identifier** (`SOURCE_ID`), **text chunk** (`CHUNK_TEXT`), and **chunk sequence number** (`CHUNK_NO`). The extracted embedding is converted into a vector format using the `VECTOR()` function and stored in the **EMBEDDING** column, which is specifically designed to handle high-dimensional vector data. Additionally, the **creation timestamp** (`CREATED_DATE`) is recorded using the system date to maintain proper tracking of data insertion.

```
INSERT INTO ERP_RAG_VECTOR_STORE
(
    VECTOR_ID,
    SOURCE_ID,
    CHUNK_TEXT,
    CHUNK_NO,
    EMBEDDING,
    CREATED_DATE
)
VALUES
(
    ERP_RAG_VECTOR_STORE_SEQ.NEXTVAL,
    p_source_id,
    v_chunk,
    v_chunk_no,
    VECTOR(v_embedding),
    SYSDATE
);
```

Figure 4.11 Storage of Embeddings

This structured storage of embeddings allows the system to efficiently perform vector-based similarity searches, such as cosine similarity or nearest neighbor search, during query processing. By maintaining a mapping between text chunks and their corresponding embeddings, the system ensures accurate retrieval of contextually relevant information. This step plays a critical role in enhancing the performance and intelligence of the Retrieval-Augmented Generation (RAG) based conversational AI system.

4.10.13 Prompt Construction

The given PL/SQL code is used to dynamically construct a JSON request body that will be sent to a Large Language Model (LLM) API for generating responses. It utilizes Oracle's JSON_OBJECT and JSON_ARRAY functions to build a structured payload. The request includes the model name (**command-r-08-2024**), which specifies the LLM to be used, and a sequence of messages that define the interaction between the system and the user.

The first message in the JSON structure is a system-level instruction, where the role is defined as "system". This part sets the behavior of the LLM by instructing it to act as an expert in a specific domain (v_domain). It also includes conditions such as responding politely to greetings and restricting answers strictly to the provided context. Additionally, it defines a fallback response when the question is outside the given context, ensuring controlled and reliable outputs.

The second message represents the user input, where the role is "user". In this section, the retrieved context (l_context) from the RAG pipeline is combined with the actual user question (p_question). The use of CHR(10) introduces line breaks for better formatting. This structured combination ensures that the LLM receives both the relevant background information and the query, enabling it to generate accurate and context-aware responses.

Finally, the entire JSON structure is returned as a CLOB and stored in the variable l_body. The max_tokens parameter limits the length of the generated response. This

constructed payload is then used as the request body for calling the LLM API, forming a critical step in connecting the Oracle APEX application with AI-based response generation.

```
SELECT JSON_OBJECT(  
    'model' VALUE 'command-r-08-2024',  
    'messages' VALUE JSON_ARRAY(  
        JSON_OBJECT(  
            'role' VALUE 'system',  
            'content' VALUE  
                'You are an expert in ' || v_domain ||  
                '. If the user greets (hi, hello, hey), respond with a friendly greeting.  
Otherwise answer ONLY using the provided context.  
If the answer is not present in the context, reply: "This question is out of context."'  
        ),  
        JSON_OBJECT(  
            'role' VALUE 'user',  
            'content' VALUE  
                'Context: ' || l_context || CHR(10) || CHR(10) ||  
                'Question: ' || p_question  
        )  
    ),  
    'max_tokens' VALUE 100  
    RETURNING CLOB  
)
```

Figure 4.12 Prompt Construction

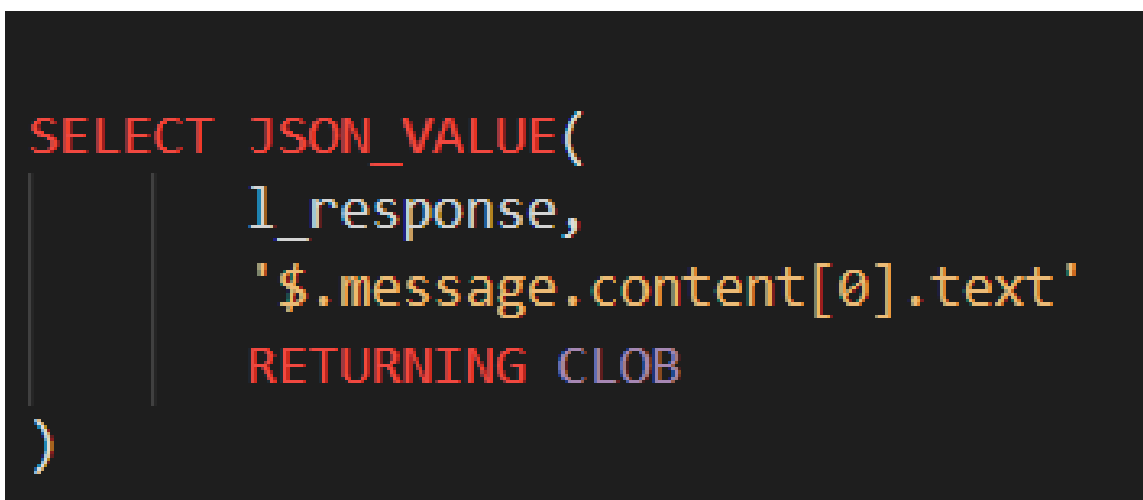
4.10.14 Response Extraction

The above PL/SQL code is used to extract the generated response from the JSON output returned by the Large Language Model (LLM) API. Once the API processes the request, it sends back a structured JSON response containing the generated text along with additional metadata. To retrieve only the required answer, the system uses Oracle's `JSON_VALUE` function to parse and extract specific data from the response.

In this implementation, the JSON path expression `'$.message.content[0].text'` is used to navigate through the response structure and locate the actual generated text. The `RETURNING CLOB` clause ensures that the extracted content is stored as a CLOB,

allowing the system to handle longer responses without size limitations. The extracted value is then assigned to the variable `p_answer`, which holds the final output to be displayed to the user.

This approach enables efficient and precise extraction of relevant information from complex JSON responses. By directly processing the API output within PL/SQL, the system maintains seamless integration with Oracle APEX and ensures that only clean and usable response data is passed to the user interface.



```
SELECT JSON_VALUE(  
    l_response,  
    '$.message.content[0].text'  
    RETURNING CLOB  
)
```

Figure 4.13 Response Extraction

4.10.15 Asynchronous Server Communication

The given JavaScript code utilizes the `apex.server.process` function in Oracle APEX to perform an asynchronous call to a backend PL/SQL process named "BOT_REPLY". This function enables seamless communication between the frontend and backend without requiring a full page refresh. The user's input (text) is passed to the server using the parameter `x01`, which acts as a temporary variable to transfer data from the client side to the server-side process.

In the success callback function, the response returned from the server (`pData`) is processed. The system logs the response to the browser console for debugging purposes and

then updates the user interface dynamically. Specifically, the chatbot's loading message is replaced with the actual response received from the backend using `pData["BOT"]`. This ensures that the user receives real-time feedback. Additionally, the input field is focused again (`userInput.focus()`), improving usability by allowing the user to continue interacting without extra clicks.

In the error callback function, the code handles scenarios where the server request fails. It logs the error details (`err.responseText`) to the console for troubleshooting and displays a fallback message ("Oops! Something went wrong.") to inform the user. The input field is again focused to maintain a smooth user experience. Overall, this implementation plays a crucial role in enabling dynamic, responsive, and interactive communication between the Oracle APEX frontend and the AI-powered backend system.

```
// Send to server (Cohere via APEX)
apex.server.process("BOT_REPLY", {
  x01: text
}, {
  success: function (pData) {
    // Replace loading text with actual reply
    console.log(pData)
    loadingMsg.textContent = pData["BOT"];
    userInput.focus();
  },
  error: function (err) {
    console.log(err.responseText)
    loadingMsg.textContent = "Oops! Something went wrong.";
    userInput.focus();
  }
});
});
```

Figure 4.14 Asynchronous Server Communication

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project successfully demonstrates the design and development of a low-code framework for conversational AI assistants by integrating Retrieval-Augmented Generation (RAG) with Large Language Models (LLMs). The system effectively addresses the limitations of traditional chatbot solutions by combining semantic retrieval with generative capabilities, enabling accurate, context-aware, and domain-specific responses. The use of a low-code platform simplifies development and allows even non-technical users to build and deploy intelligent assistants.

The implementation was carried out in two phases, where Phase-1 focused on designing an intuitive and scalable user interface using Oracle APEX, and Phase-2 focused on backend integration and functional implementation of the RAG pipeline. The successful integration of document processing, embedding generation, vector storage, retrieval mechanisms, and LLM-based response generation transformed the system into a fully functional conversational AI framework.

Overall, the project achieves its objective of providing a scalable, flexible, and user-friendly solution for building AI-powered assistants. The framework not only reduces development complexity but also ensures high performance and adaptability, making it suitable for real-world enterprise applications across various domains such as ERP, CRM, and customer support.

5.2 FUTURE WORK AND EXTENTION

The proposed system can be further enhanced by incorporating advanced features and extending its capabilities to support more complex use cases. One potential improvement is the integration of fine-tuning mechanisms for Large Language Models (LLMs), allowing the system to adapt more effectively to specific enterprise domains. Additionally, support for multimodal inputs such as images, audio, and voice interactions can be introduced to enable richer and more interactive user experiences.

Another important area of enhancement is the implementation of advanced analytics and monitoring dashboards. These dashboards can provide insights into user behavior, query patterns, system performance, and response accuracy, enabling data-driven improvements. Features such as real-time collaboration, workflow versioning, and role-based customization can also be added to support team-based development within the low-code environment.

Furthermore, the framework can be extended to support enterprise-grade requirements such as Single Sign-On (SSO), enhanced security mechanisms, and data encryption to ensure compliance and data protection. Exploring edge deployment and on-premise solutions can improve privacy and reduce latency for sensitive applications. Continuous learning mechanisms and automated evaluation pipelines can also be incorporated to enable the system to evolve and improve over time.

REFERENCES

- [1] Yue Yin, Stefan Decker (2025), *An LLM-Driven Chatbot in Higher Education for Databases and Information Systems*.
- [2] Siar Sarferaz (2025), *Implementing Generative AI into ERP Software*.
- [3] R. Yang et al. (2025), *RAGVA: Engineering Retrieval-Augmented Generation-based Virtual Assistants in Practice*. arXiv / IEEE-indexed workshops.
- [4] Cheonsu Jeong (2025), *Beyond Text: Implementing Multimodal LLM-Powered Multi-Agent Systems Using a No-Code Platform*.
- [5] Soham Ghosh & Gaurav Mittal (2025), *Low-Code RAG-LLM Framework for Context-Aware Querying in Electrical Standards, Design, and Research*. Preprint on Preprints.org.
- [6] O. Marquez Ayala & P. Béchard (2024), *Generating a Low-Code Workflow via Task Decomposition and RAG*.
- [7] Y. Cai, Z. Xing, et al. (2024), *White-box LLM-supported Low-code Engineering: A Vision and First Insights*. ACM/IEEE MODELS Companion.
- [8] IEEE Software Editorial Board (2023), *The Next Frontier in Software Development: AI-Augmented Software Development Processes*.
- [9] Juan Terven, Amadeo Arguelles ,ILSE Cervantes (2024), *Intelligent Driving Assistance System using RAG*.
- [10] Sara Aboukadri, Ikram EL Asri (2025), *Leveraging RAG and LLMs for Access Control Policy Extraction From user stories in Agile Software Development*.
- [11] LlamaIndex. [Online]. Available: <https://llamaindex.ai>
- [12] LangChain Documentation. [Online]. Available: <https://docs.langchain.com>
- [13] Oracle APEX Documentation. [Online]. Available: <https://docs.oracle.com/en/database/oracle/apex/>
- [14] OpenAI Embeddings API. [Online]. Available: <https://platform.openai.com/docs/guides/embeddings>

CONFERENCE CERTIFICATE

INTERNATIONAL CONFERENCE ON COGNITIVE INFORMATICS ENGINEERING AND TECHNOLOGY- 2026

Organised by
**VIDYAA VIKAS COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
TIRUCHENGODE, TAMILNADU, INDIA.**

In collaboration with
**OSIET , Chennai , India .
SAMARKAND STATE UNIVERSITY, SAMARKAND , UZBEKISTAN .**

Certificate of Presentation

A Low-Code Framework for Conversational AI Assistants
Leveraging Retrieval-Augmented Generation and LLMs

This is to Certify that the paper entitled
**A Low-Code Framework for Conversational AI Assistants Leveraging
Retrieval-Augmented Generation and LLMs**

Authored by
Thanigaivel G

VISTAS, Chennai

has been presented at

**International Conference on Cognitive Informatics Engineering and Technology- 2026
held on 28th & 29th March 2026 at
Vidyaa Vikas College of Engineering And Technology, (Autonomous)
Tiruchengode , Tamilnadu , India.**


Dr.K.Pooranapriya
Principal & Conference Chair


K.Janani
CEO, OSIET


Dr.Chiristo Ananth
Professor
Samarkand State University, Uzbekistan


Dr.Akhatov Akmal Rustamovich
Vice Rector (International Cooperation)
Samarkand State University, Uzbekistan